PT Motion API and Sample Program 說明

Contents

1. Function description ……..…….…… 3
2. Sample program …….....…….…..... 10
3. Appendix: Source code 14

1. Function Description

APS_pt_motion_add_point	Add points to PT buffer

Descriptions:

The properties of PT motion are continuous position profile, step velocity profile, and discontinuous acceleration profile. This function is used to add multi PT points including position and time information to the PT buffer (FIFO type). The total space of this PT buffer is 1000. Before using this function, user should use *APS_get_pt_motion_status()* to check buffer space is enough or not. If buffer space is enough, user can insert maximum 8 PT points to buffer one time by calling this function. When accessing massive amounts of PT points, it is suggested to create highest priority thread in user's program. When buffer is empty at all, the PT motion will stop automatically. The point count from *APS_get_pt_motion_status()* can provide information of accumulated finished points during PT motion. Use *APS_pt_motion_reset()* can reset the point count.

It is noted that each PT point's absolute time should increase monotonically and the minimum time interval between points is equal to the kernel's motion cycle time (for example: 1 msec in AMP-204C). It is not necessary to set first point's position and time that begin from zero. The position of first PT point should align to current axis's command position. Otherwise, the final trajectory will have offset from original pattern.

Syntax:

C/C++:

I32 APS_pt_motion_add_point (I32 Axis_ID, I32 ArraySize, F64 *PositionArray, F64 *TimeArray);

Visual Basic:

APS_pt_motion_add_point(ByVal Axis_ID As Long, ByVal ArraySizeAs Long, PositionArrayAs Double, TimeArrayAs Double) As Long

Parameters:

I32 Axis_ID:The Axis ID is from 0 to 65535.

I32 ArraySize: Total point array size. Range is from 1 to 8.

F64 *PositionArray: Position information array (unit: pulse)

F64 *TimeArray: Time information array (unit: second)

Return Values:

I32 Error code: Please refer to error code table.

Example:

Consider an example of two axis coordinated motion:

Axis 0:

Position (Count)	0	100000	100000	0	0
Time (sec)	0	2.1	4.1	6.1	8.1

Axis 1:

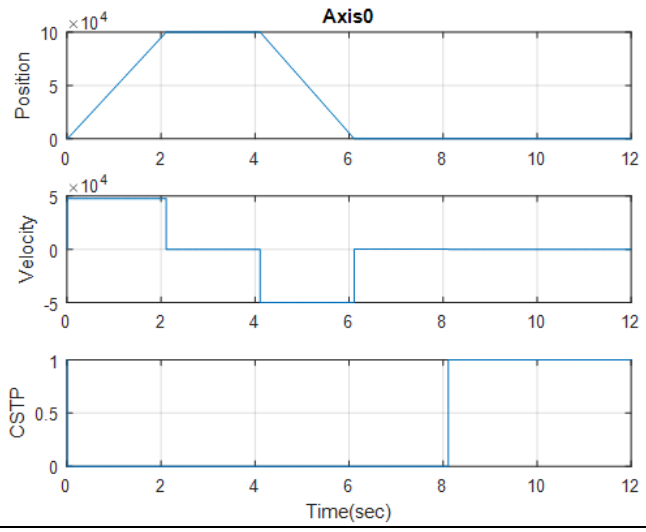
Position (Count)	0	0	100000	100000	0
Time (sec)	0	2.1	4.1	6.1	8.1

Sample Code:

```
I32 ret = 0; //Error code
U32 i = 0;
FILE *pOutput; //For sampling results
I32 Dimension = 2;
I32 ArraySize = 5;
I32 AxisID_Array[2] = {0,1};
I32 FreeSize[2];
I32 PointCount[2];
I32 State[2];
F64 Axis0PositionArray[5] = {0,100000,100000,0,0};    //Axis 0 position array.
F64 Axis0TimeArray[5] = {0.0,2.1,4.1,6.1,8.1};        //Axis 0 time array
F64 Axis1PositionArray[5] = {0,0,100000,100000,0};    //Axis 1 position array.
F64 Axis1TimeArray[5] = {0.0,2.1,4.1,6.1,8.1};        //Axis 1 time array
I32 Length = 12000;  //Total sampling data array size.
I32 PreTrgLen = 10; //The number of pre-trigger points
I32 TimeOutMs = 300000; //300 second for timeout
I32 BoardID = 0;
I32 Count = 0;
SAMP_PARAM Param;
STR_SAMP_DATA_8CH DataArr[20100]; //Sampling data array

// Sampling function configuration
ret = APS_stop_wait_sampling(BoardID);
Param.rate = 1;   // Sampling rate
Param.edge = 0;   // Rising edge
Param.level = 1;  // Trigger level
Param.trigCh = 0; // Trigger channel: Channel 0
Param.sourceByCh[0][1] = AxisID_Array[0]; // channel 0: Set axis 0
Param.sourceByCh[0][0] = 0x2;             // channel 0: Sampling command velocity
Param.sourceByCh[1][1] = AxisID_Array[0]; // channel 1: Set axis 0
Param.sourceByCh[1][0] = 0x0;             // channel 1: Sampling command position
Param.sourceByCh[2][1] = AxisID_Array[0]; // channel 2: Set axis 0
Param.sourceByCh[2][0] = 0x9;             // channel 2: Sampling motion status CSTP bit
Param.sourceByCh[3][1] = AxisID_Array[1]; // channel 3: Set axis 1
Param.sourceByCh[3][0] = 0x2;             // channel 3: Sampling command velocity
Param.sourceByCh[4][1] = AxisID_Array[1]; // channel 4: Set axis 1
Param.sourceByCh[4][0] = 0x0;             // channel 4: Sampling command position
Param.sourceByCh[5][1] = AxisID_Array[1]; // channel 5: Set axis 1
Param.sourceByCh[5][0] = 0x9;             // channel 5: Sampling motion status CSTP bit
ret = APS_set_sampling_param_ex(BoardID, &Param); //Set sampling parameters
ret = APS_wait_trigger_sampling_async_ex(BoardID, Length, PreTrgLen, TimeOutMs, DataArr ); // start
sampling function

// Create output file
pOutput = fopen( ".\\0_pt_no_thread.txt", "w" );
Sleep(1000);
```

```
// Reset PVT buffer, command position and check buffer status
for(i=0; i<2; i++)
{
      ret = APS_set_servo_on( AxisID_Array[i], 1 );
      ret = APS_set_command( AxisID_Array[i], 0 );
      ret = APS_pt_motion_reset( AxisID_Array[i] );
      ret = APS_pt_motion_get_status( AxisID_Array[i], &FreeSize[i], &PointCount[i], &State[i] );
}

// Add points to PT buffer
ret = APS_pt_motion_add_point( AxisID_Array[0], ArraySize, Axis0PositionArray, Axis0TimeArray );
ret = APS_pt_motion_add_point( AxisID_Array[1], ArraySize, Axis1PositionArray, Axis1TimeArray );

// Start PT motion
ret = APS_pt_motion_start( Dimension, AxisID_Array, 1 );
Sleep(10);

// Wait sampling process complete
while(1)
{
      ret = APS_get_sampling_count(BoardID, &Count);
      if (Count == Length)
          break;
      if (Count == -1)
          break;
      Sleep(10);
}

// Write sampling results to file
for(i=0; i<Length; i++)
{
      fprintf(pOutput, "%d %d %d %d %d %d %d\n",DataArr[i].tick, DataArr[i].data[0], DataArr[i].data[1],
DataArr[i].data[2], DataArr[i].data[3], DataArr[i].data[4], DataArr[i].data[5] );
}
fclose(pOutput);
```

Sampling Results:



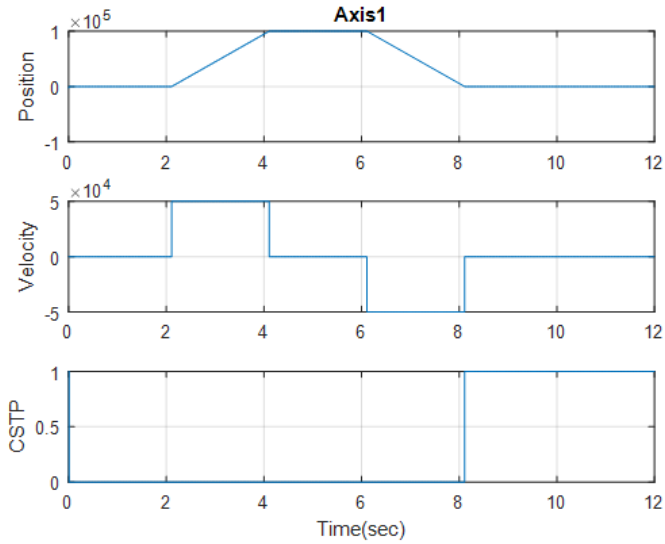


Figure 1 Sampling results of Axis 0 command position, command velocity, and motion status CSTP bit

Figure 2 Sampling results of Axis 1 command position, command velocity, and motion status CSTP bit

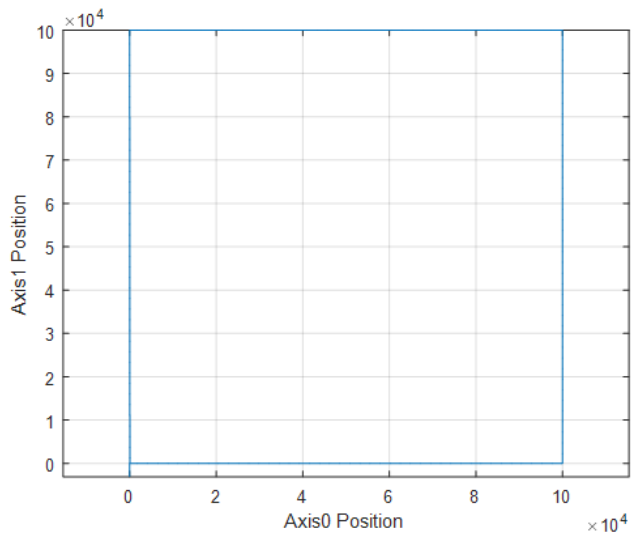


Figure 3 Trajectory of Axis 0 and Axis 1

See also:

APS_pt_motion_get_status	Get PT buffer status

Descriptions:

This function is used to get status of PT buffer. This status provides information about buffer free space, accumulated finished points, and PT motion state machine for user. Based on buffer free space information, inserting points into buffer dynamically is allowed. And the amount of accumulated finished points indicates how many points are finished in PT motion. There are two states (idle and running) defined in PT state machine. It is not allowed to start a new PT motion when state machine is not in idle state.

Syntax:

C/C++:

I32 APS_pt_motion_get_status(I32 Axis_ID, I32 *FreeSize, I32 *PointCount, I32 *State);

Visual Basic:

APS_pt_motion_get_status (ByVal Axis_ID As Long, FreeSize As Long, PointCountAs Long, StateAs Long) As Long

Parameters:

I32 Axis_ID:The Axis ID is from 0 to 65535.

I32 *FreeSize: Free buffer space. Range is from 0 to 1000; Reset by *APS_pt_motion_reset()*

F64 *PointCount: Counter for accumulated finished points; Reset by *APS_pt_motion_reset()*

F64 *State: PT motion state: 0: idle; 1: running; 2: stopping

Return Values:

I32 Error code: Please refer to error code table.

Example:

Please refer to the example in *APS_pt_motion_add_point()*

See also:

APS_pt_motion_start	Start PT motion

Descriptions:

This function is used to start or stop PT motion up to 8 axes simultaneously. Two situations are not allowed to begin a new PT motion: PT state machine is still in running/stopping state or points in PT buffer are less than two. These situation can be checked by *APS_pt_motion_get_status()*. On the other hand, several operation or situation will also result in PT motion stop immediately: PT buffer is empty, *APS_stop_move()* and *APS_emg_stop()* are used, motion IO (EMG, ALM, EL) is triggered, and so on.

Syntax:

C/C++:

I32 APS_pt_motion_start (I32 Dimension, I32 *Axis_ID_Array, I32 Enable);

Visual Basic:

APS_pt_motion_start(ByVal Dimension As Long, Axis_ID_Array As Long, ByVal Enable As Long) As Long

Parameters:

I32 Dimension:The dimension is from 1 to 8.

I32 *Axis_ID_Array: Axis ID array; its range is from 0 to 65535

I32Enable: PT motion control; 0: stop; 1: start

Return Values:

I32 Error code: Please refer to error code table.

Example:

Please refer to the example in *APS_pt_motion_add_point()*

See also:

APS_pt_motion_reset	Reset PT buffer data and point count

Descriptions:

This function is used to reset PT buffer data and point count before starting new PT motion.

Syntax:

C/C++:

I32 APS_pt_motion_reset (I32 Axis_ID);

Visual Basic:

APS_pt_motion_reset (ByVal Axis_ID As Long) As Long

Parameters:

I32Axis_ID: The Axis_ID range is from 0 to 65535

Return Values:

I32 Error code: Please refer to error code table.

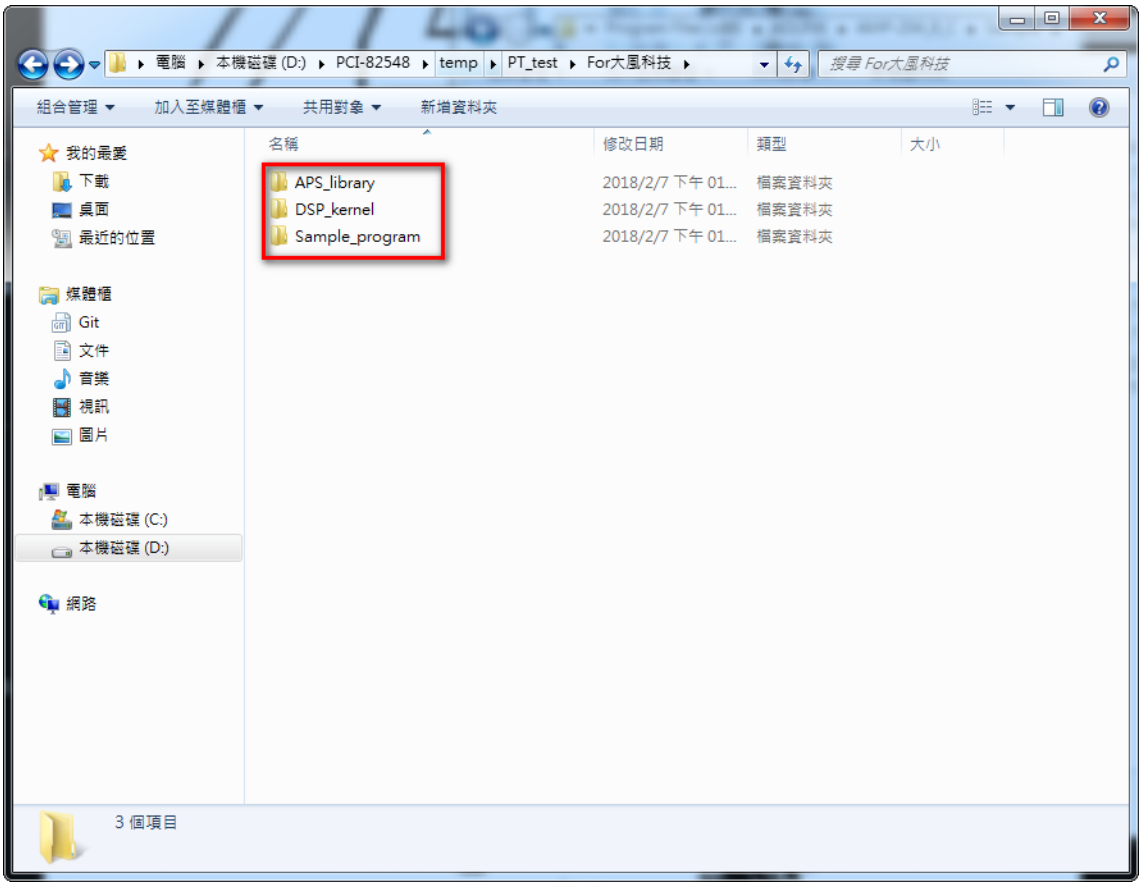
Example:

Please refer to the example in *APS_pt_motion_add_point()*

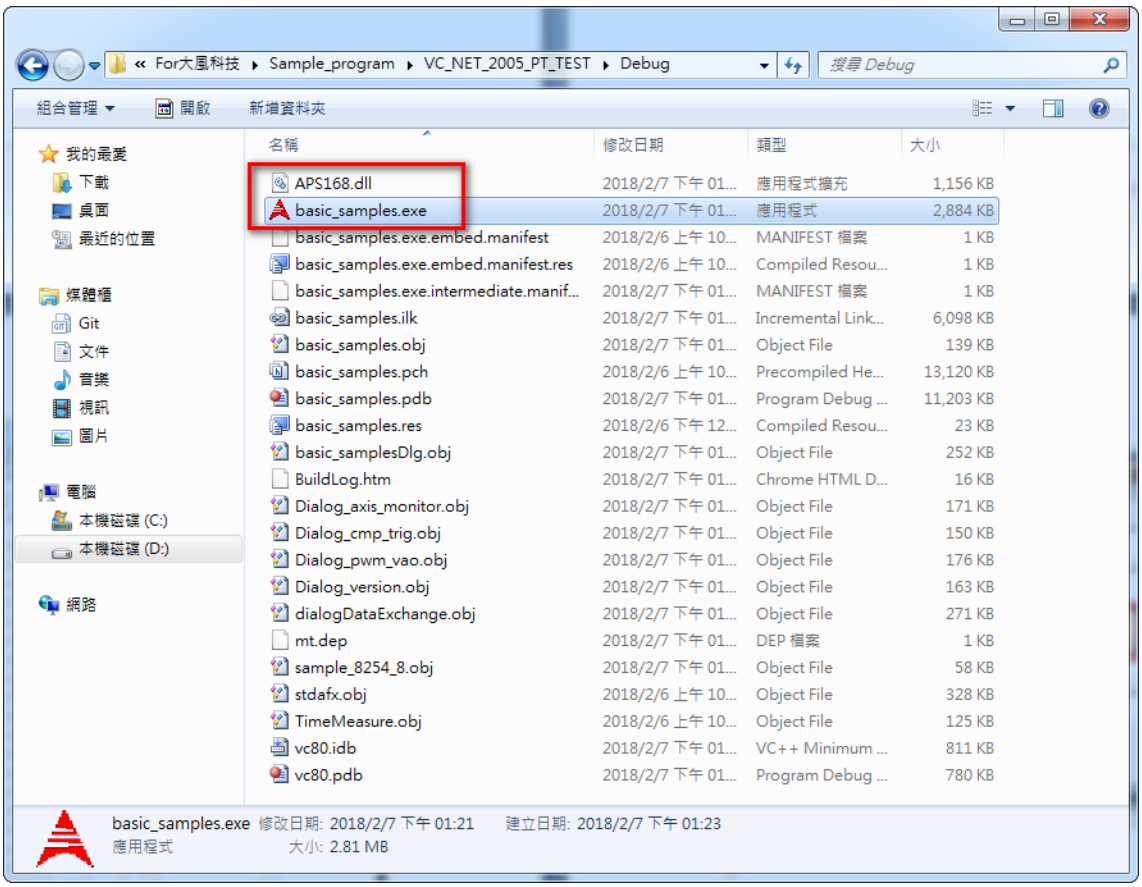
See also:

2. Sample program

解壓縮檔 **For 大風科技.rar** 後有以下資料夾，其中 **APS_library 資料夾**為所需 dll 和 header file，**DSP_kernel 資料夾**則為 AMP-204C/208C 的 DSP kernel file，而 **Sample_program 資料夾**則為 sample program for PT motion。

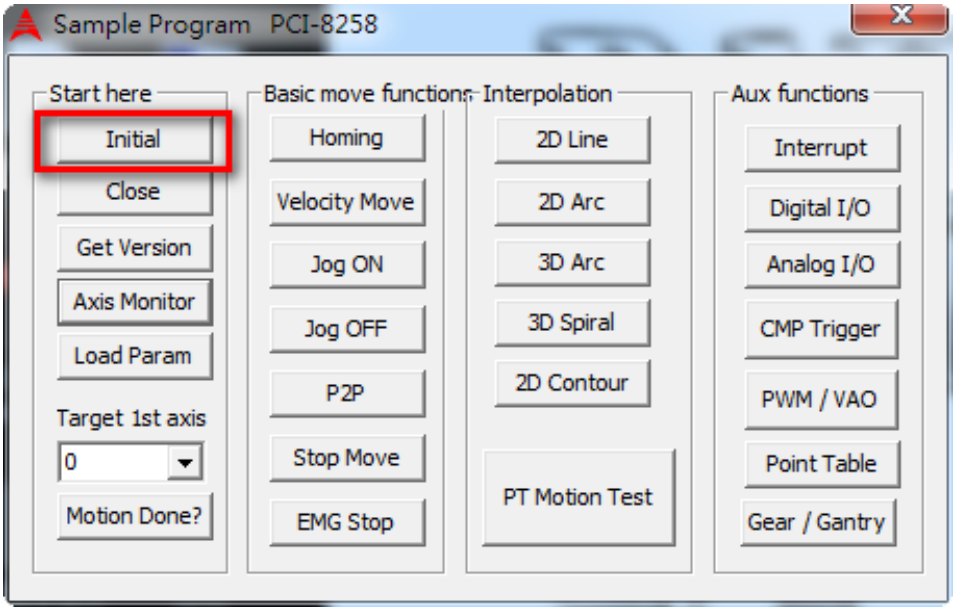


Sample program 內有測試 PT motion 的 source code (可參考後面 Apendix)，也可使用已編譯好執行檔 **basic_samples.exe** for PT motion test 如下所示。

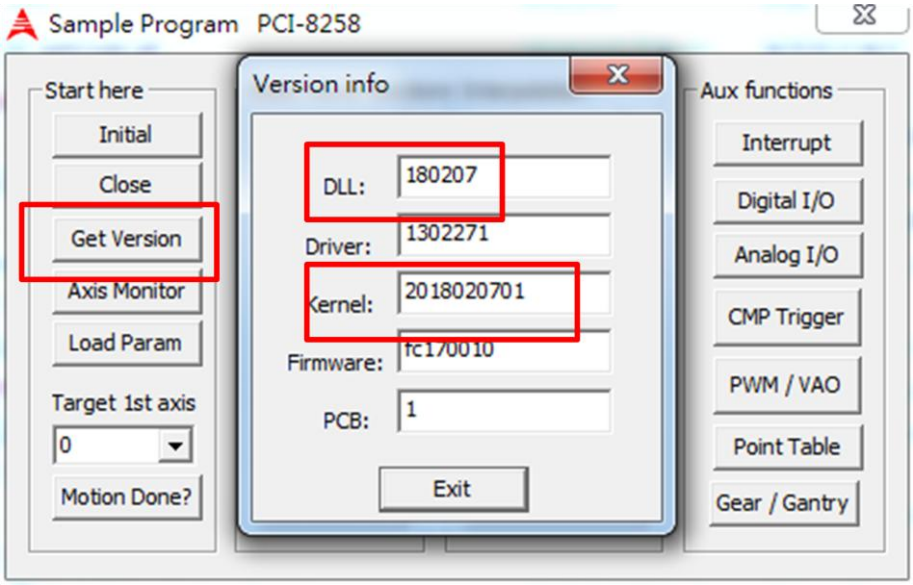


以下將介紹 Sample program 使用方式。

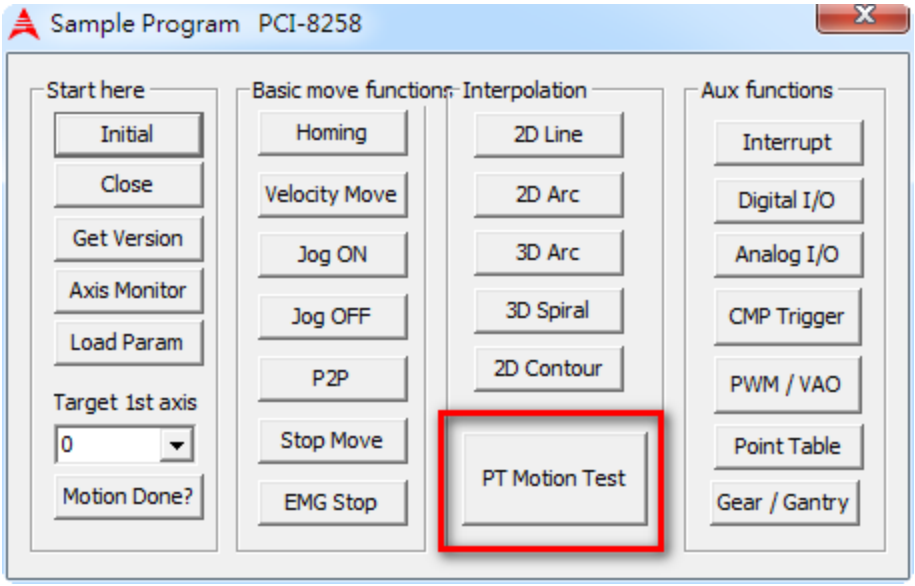
<u>Step 1:</u> 執行檔 **basic_samples.exe** 後要 Initialize APS library 如下所示



Step 2: 確認軟體版本



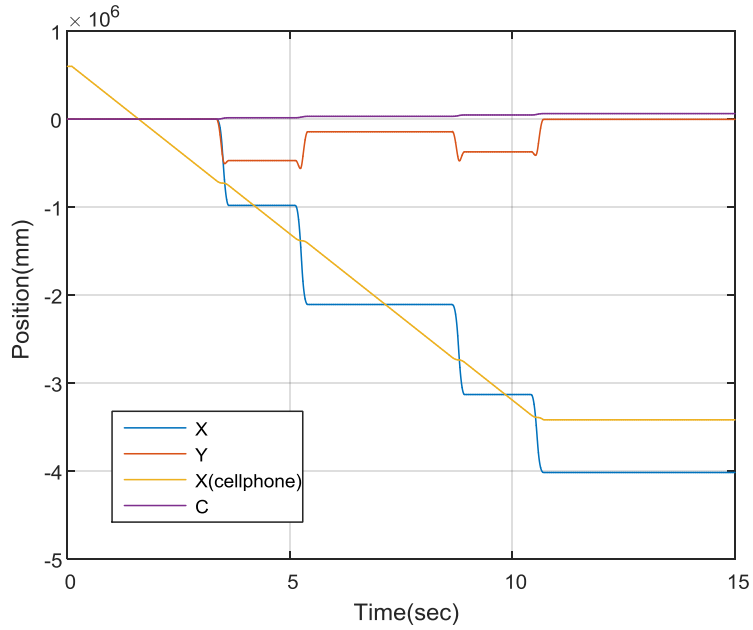
Step 3: Start PT motion，其中會根據 **PT_Test_Raw_Data.txt** 裡面四軸(X, Y, X(手機), C)給予的點位執行 PT motion，每個點位間隔為 1 msec。

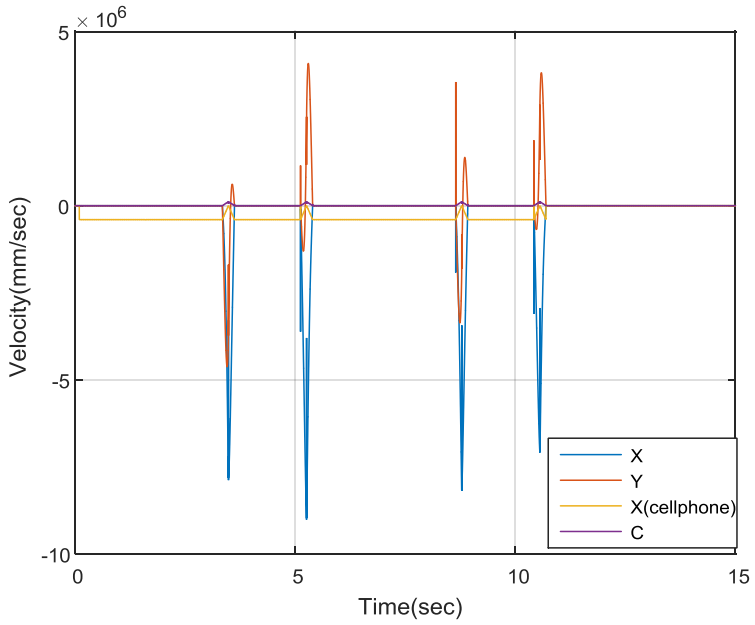


Step 4: PT motion 完成後會產生以下檔案作為觀察結果使用

(1) **PT_Test_Sampling_Result.txt**: 使用取樣功能(1 msec)所得的 4 個軸的資料，排列方式為 time tick, Axis0 command velocity, Axis0 command position, Axis1

command velocity, Axis1 command position, Axis2 command velocity, Axis2 command position, Axis3 command velocity, Axis3 command position，使用繪圖軟體可得結果如下圖所示





(2) **PT_Test_Status.txt**: 此檔記錄了 buffer freesize 和 point count，其資料排列方式為 Axis0 buffer freesize, Axis1 buffer freesize, Axis2 buffer freesize, Axis3 buffer freesize, Axis0 point count , Axis1 point count, Axis2 point count, Axis3 point count

3. Appendix: Source code

以下為 Sample program 有關 PT motion test 的 source code:

```
/**************************************************
Sample Program: PT Motion for 大風科技
**************************************************/
// Start PT motion process
void Cbasic_samplesDlg::OnBnClickedButton22()
{
	// TODO: Add your control notification handler code here
	I32 ret = 0;
	DWORD   dwThreadId1;
	DWORD   dwThreadId2;
	HANDLE  hThread1;
	HANDLE  hThread2;
	U32 i = 0;
	I32 AxisID_Array[4] = {0,1,2,3};
	I32 Dimension = 4;
	I32 FreeSize[4] = {0,0,0,0};
	I32 PointCount[4] = {0,0,0,0};
	I32 State[4] = {0,0,0,0};
	PT_Start_Flag = 0;
	PT_Stop_Flag = 0;

	// Set Servo on, set command,and reset PT buffer and point coount
	for(i=0; i<4; i++)
	{
		ret = APS_set_servo_on( AxisID_Array[i], 1 );
		ret = APS_set_command( AxisID_Array[i], 0 );
		ret = APS_pt_motion_reset( AxisID_Array[i] );
		ret = APS_pt_motion_get_status( AxisID_Array[i], &FreeSize[i], &PointCount[i], &State[i] );
	}

	// Create 1st thread for sampling PT motion results
	hThread1 = CreateThread(
		NULL,                   // default security attributes
		0,                      // use default stack size
		MyThreadFunction4,      // thread function name
		0,                      // argument to thread function
		0,                      // use default creation flags
		&dwThreadId1);          // returns the thread identifier

	// Wait sampling configuration done
	Sleep(100);

	// Create 2nd thread to insert points into PT buffer
	hThread2 = CreateThread(
		NULL,                   // default security attributes
		0,                      // use default stack size
		MyThreadFunction3,      // thread function name
		0,                      // argument to thread function
		0,                      // use default creation flags
		&dwThreadId2);          // returns the thread identifier

	// Set 2nd thread priority to high
```

```
	SetThreadPriority( hThread2, THREAD_PRIORITY_HIGHEST );

	// Wait 2nd thread insert points until buffer freesize is less than 200
	while(1)
	{
		APS_pt_motion_get_status( AxisID_Array[0], &FreeSize[0], &PointCount[0], &State[0] );
		APS_pt_motion_get_status( AxisID_Array[1], &FreeSize[1], &PointCount[1], &State[1] );
		APS_pt_motion_get_status( AxisID_Array[2], &FreeSize[2], &PointCount[2], &State[2] );
		APS_pt_motion_get_status( AxisID_Array[3], &FreeSize[3], &PointCount[3], &State[3] );

		// Check PT buffer freesize is less than 200 points or not
		if( (FreeSize[0]<200) && (FreeSize[1]<200) && (FreeSize[2]<200) && (FreeSize[3]<200))
		{
			// Start 4 axis PT motion simutaneously
			ret = APS_pt_motion_start( Dimension, AxisID_Array, 1 );
			if(ret)
			{
				PT_Stop_Flag = 1;
				MessageBox( "PT stop" );
			}
			else
			{
				PT_Start_Flag = 1;
				MessageBox( "PT start" );
			}
			break;
		}
	}
}
```

```
// Thread to insert points into PT buffer
DWORD WINAPI MyThreadFunction3( LPVOID lpParam )
{
	U32 i = 0;
	U32 j = 0;
	I32 FreeSize[4] = {0,0,0,0};
	I32 PointCount[4] = {0,0,0,0};
	I32 State[4] = {0,0,0,0};
	I32 ret = 0;
	U32 usedPoint[4] = {0,0,0,0};
	U32 residualPoint[4] = {0,0,0,0};
	I32 AxisID_Array[4] = {0,1,2,3};
	U32 temp = 0;
	FILE *pIn;
	U32 totalPoint = 0;
	U32 MAX_POINT_SIZE = 15000;
	F64 L = 0;
	U32 totalPt[4] = {0,0,0,0};
	F64 P[4][15000];
	F64 T[4][15000];
	FILE *pOutput;
	pOutput = fopen( ".\\PT_Test_Status.txt", "w" );

	// Read 大風科技raw data
```

```
    pIn = fopen( ".\\PT_Test_Raw_Data.txt", "r" );
    if(pIn == NULL)
    {
        printf("fopen fail  \n");
        return 0;
    }

    while(1)
    {
        fscanf(pIn,"%lf %lf %lf %lf",&P[0][totalPoint], &P[1][totalPoint],
&P[2][totalPoint], &P[3][totalPoint]);
        totalPoint++;

        if(feof(pIn))
            break;
    }
    totalPt[0] = totalPoint; // total points of Axis 0 is 10598
    totalPt[1] = totalPoint; // total points of Axis 1 is 10598
    totalPt[2] = totalPoint; // total points of Axis 2 is 10598
    totalPt[3] = totalPoint; // total points of Axis 3 is 10598

    // Assign absolute time: 0.001 sec (1 ms) interval per point
    T[0][0] = 0;
    T[1][0] = 0;
    T[2][0] = 0;
    T[3][0] = 0;
    for(i=0; i< totalPoint-1; i++)
    {
        T[0][i+1]= 0.001 + T[0][i];
        T[1][i+1]= 0.001 + T[1][i];
        T[2][i+1]= 0.001 + T[2][i];
        T[3][i+1]= 0.001 + T[3][i];
    }

    // Align current command position to first point's position
    ret = APS_set_command( 0, P[0][0] );
    ret = APS_set_command( 1, P[1][0] );
    ret = APS_set_command( 2, P[2][0] );
    ret = APS_set_command( 3, P[3][0] );

    // Stert to insert points into buffer
    while(1)
    {
        for(i=0; i< 4; i++)
        {
            // Update residualPoint that nedd to be inserted into PT buffer; the usedPoint
is the points that already inserted into buffer
            residualPoint[i] = totalPt[i] - usedPoint[i];

            // Read buffer freesize
            ret = APS_pt_motion_get_status( AxisID_Array[i], &FreeSize[i], &PointCount[i],
&State[i] );

            // Check buffer freesize is larger than 8 or not
            if(FreeSize[i] >= 8)
            {
                if(residualPoint[i]>=8) //  The max. points that can insert to buffer every
cycle time is 8 points.
```

```
                    {
                         ret = APS_pt_motion_add_point( AxisID_Array[i], 8,
&P[i][usedPoint[i]],  &T[i][usedPoint[i]] );
                         if(ret ==0)
                              usedPoint[i] += 8;
                    }
                    else
                    {
                         ret = APS_pt_motion_add_point( AxisID_Array[i], residualPoint[i],
&P[i][usedPoint[i]],  &T[i][usedPoint[i]] );
                         if(ret ==0)
                              usedPoint[i] += residualPoint[i];
                    }
               }
               else // buffer freesize is less than 8 or not
               {
                    if(FreeSize[i]>=residualPoint[i])
                    {
                         ret = APS_pt_motion_add_point( AxisID_Array[i], residualPoint[i],
&P[i][usedPoint[i]],  &T[i][usedPoint[i]] );
                         if(ret ==0)
                              usedPoint[i] += residualPoint[i];
                    }
                    else
                    {
                         ret = APS_pt_motion_add_point( AxisID_Array[i], FreeSize[i],
&P[i][usedPoint[i]],  &T[i][usedPoint[i]] );
                         if(ret ==0)
                              usedPoint[i] += FreeSize[i];
                    }
               }
          }

          // Update buffer freesize and point count (Count for finished point data)
          fprintf(pOutput,"%d %d %d %d %d %d %d %d\n",FreeSize[0], FreeSize[1], FreeSize[2],
FreeSize[3], PointCount[0], PointCount[1],PointCount[2], PointCount[3] );

          // Check 4 axis motion status (CSTP) is done or not
          if(PT_Start_Flag == 1)
          {
               if( (APS_motion_status(0)&0x1) && (APS_motion_status(1)&0x1) &&
(APS_motion_status(2)&0x1) && (APS_motion_status(3)&0x1 ) )
                    break;
          }

          // Stop PT motion
          if(PT_Stop_Flag == 1)
               break;
     }

     // Update final buffer freesize and point count
     APS_pt_motion_get_status( AxisID_Array[0], &FreeSize[0], &PointCount[0], &State[0] );
     APS_pt_motion_get_status( AxisID_Array[1], &FreeSize[1], &PointCount[1], &State[1] );
     APS_pt_motion_get_status( AxisID_Array[2], &FreeSize[2], &PointCount[2], &State[2] );
     APS_pt_motion_get_status( AxisID_Array[3], &FreeSize[3], &PointCount[3], &State[3] );

     fprintf(pOutput,"%d %d %d %d %d %d %d %d\n",FreeSize[0], FreeSize[1], FreeSize[2],
FreeSize[3], PointCount[0], PointCount[1],PointCount[2], PointCount[3] );
```

```
	fclose(pOutput);


	// Set servo off
	for(i=0; i<4; i++)
	{
		ret = APS_set_servo_on( AxisID_Array[i], 0 );
	}

	return 0;
}
```

```
// Thread for sampling results
DWORD WINAPI MyThreadFunction4( LPVOID lpParam )
{
	I32 ret = 0; //Error code
	U32 i = 0;
	FILE *pOutput; //For sampling results
	I32 AxisID_Array[4] = {0,1,2,3};
	I32 Length = 15000;  //Total sampling data array size.
	I32 PreTrgLen = 100; //The number of pre-trigger points
	I32 TimeOutMs = 3000000; //300 second for timeout
	I32 BoardID = 0;
	I32 Count = 0;
	SAMP_PARAM Param;
	STR_SAMP_DATA_8CH DataArr[20100]; //Sampling data array

	// Sampling function configuration (Can refer APS library manual)
	ret = APS_stop_wait_sampling(BoardID);
	Param.rate = 1;   // Sampling rate
	Param.edge = 1;   // Rising edge
	Param.level = -1;  // Trigger level
	Param.trigCh = 4; // Trigger channel: Channel 0
	Param.sourceByCh[0][1] = AxisID_Array[0]; // channel 0: Set axis 0
	Param.sourceByCh[0][0] = 0x2;             // channel 0: Sampling command velocity
	Param.sourceByCh[1][1] = AxisID_Array[0]; // channel 1: Set axis 0
	Param.sourceByCh[1][0] = 0x0;             // channel 1: Sampling command position
	Param.sourceByCh[2][1] = AxisID_Array[1]; // channel 2: Set axis 1
	Param.sourceByCh[2][0] = 0x2;             // channel 2: Sampling command velocity
	Param.sourceByCh[3][1] = AxisID_Array[1]; // channel 3: Set axis 1
	Param.sourceByCh[3][0] = 0x0;             // channel 3: Sampling command position
	Param.sourceByCh[4][1] = AxisID_Array[2]; // channel 4: Set axis 2
	Param.sourceByCh[4][0] = 0x2;             // channel 4: Sampling command velocity
	Param.sourceByCh[5][1] = AxisID_Array[2]; // channel 5: Set axis 2
	Param.sourceByCh[5][0] = 0x0;             // channel 5: Sampling command position
	Param.sourceByCh[6][1] = AxisID_Array[3]; // channel 5: Set axis 3
	Param.sourceByCh[6][0] = 0x2;             // channel 5: Sampling command velocity
	Param.sourceByCh[7][1] = AxisID_Array[3]; // channel 5: Set axis 3
	Param.sourceByCh[7][0] = 0x0;             // channel 5: Sampling command position
	ret = APS_set_sampling_param_ex(BoardID, &Param); //Set sampling parameters
	ret = APS_wait_trigger_sampling_async_ex(BoardID, Length, PreTrgLen, TimeOutMs, DataArr ); // start
sampling function

	// Create output file
```

```
	pOutput = fopen( ".\\PT_Test_Sampling_Result.txt", "w" );
	Sleep(1000);

	// Wait sampling process complete
	while(1)
	{
		ret = APS_get_sampling_count(BoardID, &Count);

		if (Count == Length)
			break;

		if (Count == -1)
			break;

		Sleep(10);
	}

	// Write sampling results to file
	for(i=0; i<Length; i++)
	{
		fprintf(pOutput, "%d %d %d %d %d %d %d %d %d\n",DataArr[i].tick, DataArr[i].data[0],
DataArr[i].data[1], DataArr[i].data[2], DataArr[i].data[3], DataArr[i].data[4], DataArr[i].data[5],
DataArr[i].data[6], DataArr[i].data[7] );
	}
	fclose(pOutput);

	return 0;
}
```